

Centro Regional Universitario Córdoba IUA

Maestría en Ciberdefensa



Trabajo Práctico N°1

MD5crypt

Módulo:

- Criptografía

Alumno:

- Ing. Vietto Herrera Santiago

Docentes:

- Ing. Montes Miguel

25 de Abril de 2026

Córdoba - Argentina

Indice

Desarrollo	1
1. Analisis del Algoritmo	1
1.1. Código de MD5crypt	1
1.1.1. Función Encrypt	1
1.1.2. Función keyED	2
1.2. Lógica interna	3
1.2.1. Cifrado de Vigenère	3
1.2.2. Problemas en la implementación	4
2. Solución del desafío	6
2.1. Análisis del problema	6
2.2. Implementación de la solución	7
2.2.1. Conexión con el servidor, obtención del desafío y decodificación Base64	7
2.2.2. Eliminación de clave auxiliar interna y reducción del texto cifrado	8
2.2.3. Obtención de los posibles bytes válidos de la clave reducida.	9
2.2.4. Obtención de la clave reducida y descifrado del texto claro	10
2.2.5. Envío de la respuesta al servidor	13

Desarrollo

1. Analisis del Algoritmo

1.1. Código de MD5crypt

Como primera medida, tenemos ADOdb, la cual es una biblioteca que se utiliza para conexiones de bases de datos en sistemas en PHP. El siguiente enlace da acceso al sitio de github de la biblioteca, el cual posee el código php del algoritmo de cifrado MD5crypt:

- <https://github.com/ADOdb/ADOdb/blob/master/session/crypt.inc.php>

El algoritmo se define en una clase MD5crypt la cual agrupa funciones para cifrar, descifrar y generar una contraseña aleatoria. A continuación, se analiza cada una de ellas.

1.1.1. Función Encrypt

La función tiene como objetivo cifrar un texto claro utilizando una clave secreta generada.

1) Como se observa en la siguiente imagen, dicha función recibe como parámetro dos argumentos de tipo string:

- *\$txt*: Texto claro que se quiere cifrar.
- *\$key*: Clave para proteger el texto claro.

```
function Encrypt($txt,$key)
{
    $encrypt_key = md5(rand(0,32000));
    $ctr=0;
    $tmp = "";
    for ($i=0;$i<strlen($txt);$i++)
    {
        if ($ctr==strlen($encrypt_key)) $ctr=0;
        $tmp.= substr($encrypt_key,$ctr,1) .
            (substr($txt,$i,1) ^ substr($encrypt_key,$ctr,1));
        $ctr++;
    }
    return base64_encode($this->keyED($tmp,$key));
}
```

2) Dentro de la función tenemos *\$encrypt_key*, en donde se genera una clave auxiliar interna y aleatoria, básicamente un número aleatorio entre 0 y 32000, y luego se le aplica una función de md5, el cual devuelve una representación hexadecimal del hash en una cadena de 32 caracteres del conjunto {0,1,2,3,4,5,6,7,8,9,a,b,c,d,e,f}.

3) Se crea un contador en 0 en la variable *\$ctr* para poder recorrer *\$encrypt_key*. Y por otro lado, se crea *\$tmp*, una variable vacía donde se va a ir armando un texto intermedio, pero que no es todavía el cifrado final.

4) El bucle for recorre caracter por caracter (cada *\$i* entre 0 y la longitud del texto original), el texto claro original almacenado en *\$txt*. Por ejemplo, si el texto claro es "hola", entonces recorre "h", luego "o", y así sucesivamente. En cada recorrido se ejecuta lo siguiente:

- Se verifica si el contador `$ctr` llegó al final de `$encrypt_key`. Teniendo en cuenta que `$encrypt_key` tiene 32 caracteres, si el texto claro es más largo que 32 caracteres, el contador vuelve a cero, y reutiliza la clave auxiliar desde el principio.
- Luego, en `$tmp` se concatenan dos cosas:
 - Un carácter de la clave auxiliar, que está en la posición `$ctr`.
 - El resultado de una operación XOR entre el carácter de la clave auxiliar anterior de la posición `$ctr`, con el carácter del texto original de la posición actual `$i`.

Entonces, siguiendo con el ejemplo del texto original “hola”, tenemos que el primer carácter del texto original es “h”, y por ejemplo el carácter actual de la clave auxiliar es “a”, en este caso en `$tmp` se guardará: `a + (a XOR h)`. Es decir, por cada carácter del texto original, `$tmp` va a guardar dos caracteres. Como tenemos “hola” en este caso, `$tmp` va a guardar 8 caracteres.

Además, la operación XOR se realiza entre los códigos ASCII correspondientes a cada carácter, tanto del texto claro original como el de la clave intermedia. Por ejemplo:

`'A' ^ 'b' = 65 ^ 98 = 35 = '#'`

- Terminada la operación anterior, el contador del bucle for avanza para tratar el siguiente carácter del texto original, y por consiguiente de `$encrypt_key`.

5) Terminado el bucle for, mediante la función `keyED`, se cifra el resultado de `$tmp`, utilizando la clave real `$key`. La longitud del texto cifrado es siempre el doble que la longitud del texto claro. Finalmente se le aplica al texto cifrado una función `base64_encode`, lo cual convierte el resultado final en Base64. Esto se hace así, porque la operación XOR no puede generar caracteres no imprimibles, entonces Base64 los transforma o cifra de alguna manera en un texto relativamente seguro.

1.1.2. Función keyED

La siguiente función, básicamente aplica el mismo procedimiento que la función `Encrypt`, con ligeras diferencias.

1) Como podemos observar en la imagen, la función recibe por parámetro dos argumentos:

- `$txt`: Es el texto intermedio `$tmp` generado en la función `Encrypt`, que aún no está cifrado.
- `$encrypt_key`: La clave secreta original, provista por el usuario, que se quiere utilizar para cifrar el texto claro original.

```
function keyED($txt,$encrypt_key)
{
    $encrypt_key = md5($encrypt_key);
    $ctr=0;
    $tmp = "";
    for ($i=0;$i<strlen($txt);$i++){
        if ($ctr==strlen($encrypt_key)) $ctr=0;
        $tmp.= substr($txt,$i,1) ^ substr($encrypt_key,$ctr,1);
        $ctr++;
    }
    return $tmp;
}
```

2) Internamente, en una nueva variable *\$encrypt_key* se almacena la aplicación de la función md5 al valor de la clave secreta, obteniendo así una nueva cadena hexadecimal de 32 caracteres. Luego, se crea el contador para recorrer la nueva clave interna generada, y una variable vacía para almacenar el texto ya cifrado, resultante del bucle for.

3) El bucle for recorre caracter por caracter el contenido del texto claro original.

- Se compara nuevamente si el texto claro es más largo que la nueva clave interna generada, entonces el contador vuelve a 0 para utilizar la clave desde el principio.
- En este caso, en la variable *\$tmp* se almacena el resultado de una operación XOR entre el carácter de la nueva clave interna auxiliar de la posición *\$ctr*, con el carácter del texto claro original de la posición actual *\$i*.
- Terminada la operación anterior, el contador del bucle for avanza para tratar el siguiente carácter del texto original, y por consiguiente de *\$encrypt_key*.

4) Terminado el bucle for, se obtiene el resultado de *\$tmp*, el cual representa el texto claro original cifrado con la clave secreta *\$key*.

1.2. Lógica interna

El algoritmo MD5crypt equivale a aplicar el cifrado de Vigenère dos veces, con claves de 32 caracteres compuestas por dígitos hexadecimales.

- En la primera aplicación, la aplicación interna, se utiliza una clave aleatoria, y se produce un nuevo texto que consiste en preceder cada carácter del texto cifrado por el carácter correspondiente de la clave.
- En la segunda aplicación, la aplicación externa, se utiliza la clave elegida por el usuario, previamente procesada por md5.

1.2.1. Cifrado de Vigenère

El cifrado de Vigenère es un cifrado que utiliza una clave repetida (rotación) para ir desplazando cada carácter del mensaje.

La fórmula de cifrado es la siguiente, sea un mensaje M de longitud n, y una clave K de longitud r:

$$\begin{aligned}
 K &= k_0, k_1, \dots, k_{r-1} \\
 M &= m_0, m_1, \dots, m_{n-1} \\
 C = E_k(M) &= c_0, c_1, \dots, c_{n-1} \\
 c_i &= m_i + k_{(i \bmod r)} \bmod 27, i = 0, \dots, n-1
 \end{aligned}$$

En donde:

- K: Posición del carácter de la clave.
- M: Posición del carácter del texto claro.
- C: Posición del carácter cifrado.

Cada carácter de la clave indica cuánto se desplaza la letra del texto, y se utiliza posiciones del alfabeto: A = 0, B = 1, C = 2, . . . , Z = 26.

A) Por ejemplo, tenemos lo siguiente:

- Texto claro: HOLA
- Clave: SOL

B) Como la clave es más corta que el texto, se repite:

- HOLA
- SOLS

C) Para aplicar el cifrado, convertimos a números:

H = 7

O = 14

L = 11

A = 0

S = 18

O = 14

L = 11

S = 18

D) Se aplica $C = (M + K) \bmod 27$:

$H + S = 7 + 19 = 26 = Z$

$O + O = 15 + 15 = 30 = 30 \bmod 27 = 3 = D$

$L + L = 11 + 11 = 22 = V$

$A + S = 0 + 19 = 19 = S$

Resultando el texto cifrado en: ZDVS

Las repeticiones en el texto cifrado permiten estimar el periodo de la clave. Además, conociendo la longitud de la clave, puede realizarse análisis de frecuencia sobre todos los caracteres a los que se les ha aplicado la misma sustitución.

1.2.2. Problemas en la implementación

Analizando en profundidad el esquema, nos encontramos con que el mismo tiene varios problemas:

1) En la función Encrypt, cuando tenemos `$encrypt_key = md5(rand(0,32000));`, se indica que la clave interna auxiliar sólo puede salir de un número del 0 al 32000 incluidos, es decir, solamente hay 32001 claves posibles, en donde para una computadora moderna probar esa cantidad de posibilidades es muy poco, facilitando los ataques de fuerza bruta contra dicha clave, sin importar que después se aplique md5, porque el valor original viene de un conjunto muy chico.

2) Los posibles valores de los bytes de las claves auxiliares no están distribuidos de manera uniforme, ya que el resultado de la función md5 se representa en texto hexadecimal, por lo que hay 16 valores posibles: 0 1 2 3 4 5 6 7 8 9 a b c d e f. Es decir, aunque un byte podría tener 256 valores posibles, cada carácter de la clave auxiliar sólo puede ser uno de 16 valores, lo que reduce mucho la aleatoriedad de la clave.

3) La clave auxiliar interna se puede eliminar fácilmente, haciendo un XOR de los caracteres pares con los impares. De esta forma obtenemos un nuevo texto cifrado, que resulta de aplicar un cifrado de Vigenère con una clave de 16 caracteres. A continuación se demuestra:

- Texto claro: $P_1 P_2 P_3 P_4 \dots P_n$
- Texto cifrado: $C_1 C_2 C_3 C_4 \dots C_{2n}$
- Clave interna: $I_1 I_2 I_3 I_4 \dots I_{32}$
- Clave externa: $O_1 O_2 O_3 O_4 \dots O_{32}$

Entonces:

$$\begin{aligned} C_1 &= I_1 \oplus O_1 \\ C_2 &= P_1 \oplus I_1 \oplus O_2 \\ C_1 \oplus C_2 &= P_1 \oplus I_1 \oplus O_2 \oplus I_1 \oplus O_1 = P_1 \oplus O_2 \oplus O_1 \end{aligned}$$

Podemos armar así un nuevo texto cifrado, $C'_1 C'_2 C'_3 C'_4 \dots C'_n$, donde

$$C'_i = P_i \oplus K_i$$

y

$$K_i = O_{2(i \bmod 32)} \oplus O_{2(i \bmod 32)+1}$$

El texto cifrado tiene la longitud del mensaje original, y la clave de cifrado tiene ahora 16 caracteres.

4) Los valores de esa clave (desconocida) de 16 caracteres, tampoco tienen distribución uniforme, es decir, no es completamente aleatoria, sino que se corresponden al XOR de dos dígitos hexadecimales, y como los caracteres hexadecimales solo pueden ser 0-9 y a-f, los resultados posibles siguen teniendo restricciones. Entonces, la clave no puede tomar cualquier valor libremente, tiene patrones y limitaciones, lo cual ayuda a un atacante, porque reduce el número de posibilidades que tiene que probar.

5) Si no se conoce nada del texto claro, se puede resolver como un Vigenère con longitud de clave conocida, que es 16 caracteres. Entonces se pueden aplicar análisis estadísticos o pruebas por bloques, intentando deducir la clave a partir de patrones del idioma o del formato esperado del mensaje. Por ejemplo, si el texto claro es un correo electrónico, probablemente tenga palabras, espacios, saludos, signos o estructura típica de un mail.

6) Si se conoce parte del texto claro, basta con hacer el XOR del texto claro conocido con la porción adecuada del texto cifrado para recuperar esa parte de la clave. Por ejemplo, si se sabe que el mensaje empieza con algo como "Hola", o que contiene una dirección de correo, un dominio, un saludo o una frase esperada, eso puede alcanzar para reconstruir parte de la clave. Y como la clave se repite, recuperar algunos caracteres puede ayudar a descifrar muchas posiciones más del mensaje.

2. Solución del desafío

2.1. Análisis del problema

Un servidor proporciona algo que parece ilegible, codificado en Base64, básicamente un mensaje que fue cifrado en el esquema MD5Crypt siguiendo el procedimiento analizado en la sección [1.1](#). Por ende, el objetivo consiste en recuperar el texto claro o mensaje original, demostrando que es posible romper dicho esquema de cifrado, al aprovechar sus debilidades conocidas. El mensaje original es texto ASCII, y tiene un formato de correo electrónico dirigido a un email registrado, en este caso el del alumno que lleva a cabo el presente trabajo práctico.

Por un lado, tenemos que Base64 no es cifrado, ya que solo sirve para representar bytes como texto visible y transportable. Es decir, convierte datos binarios en caracteres como letras, números, etc. Por lo que es posible decodificar Base64 para recuperar los bytes cifrados reales.

Pero la debilidad principal está en cómo el algoritmo arma el texto cifrado, ya que por cada carácter del mensaje original, el algoritmo genera la siguiente estructura similar:

- $K + (M \text{ XOR } K)$

En donde:

- K: Caracter de la clave auxiliar interna.
- M: Caracter del mensaje original.

Y dado el análisis anterior, si uno hace XOR entre esas dos partes, la clave auxiliar se cancela porque $K \text{ XOR } K = 0$. Entonces la clave auxiliar, que supuestamente agrega seguridad, puede ser anulada por la propia forma en que el mensaje fue construido.

- $K \text{ XOR } (M \text{ XOR } K) = M$

Por ende, una vez eliminada la clave interna, el cifrado deja de ser el esquema original completo y se transforma en algo mucho más simple. Queda un cifrado parecido a Vigenère, pero usando XOR en lugar de desplazamientos alfabéticos. Conceptualmente quedaría algo como:

- Texto claro XOR clave reducida = texto cifrado

Y sabemos que la clave efectiva tiene una longitud conocida de 16 bytes, lo cual facilita mucho el análisis al no tener que atacar un algoritmo desconocido o complejo, sino resolver un cifrado con clave corta y repetida.

Además, la consigna da una ventaja muy importante. Nos dice que el mensaje claro es un correo electrónico dirigido al email declarado, lo que significa que dicho email aparece dentro del mensaje. Por ejemplo, si el correo declarado es `santiagovietto5@gmail.com`, es razonable asumir que el mismo aparece en el texto claro, por ejemplo en el campo "To: `santiagovietto5@gmail.com`". Resultando así en tener una parte conocida del mensaje original, y en un cifrado con XOR, si conocemos el texto claro y tenemos el texto cifrado, podemos recuperar la clave:

- texto cifrado XOR texto claro = clave

Como no se sabe exactamente en qué posición aparece el email dentro del mensaje, se prueba ubicarlo en distintas posiciones del texto cifrado. En donde para cada posible posición, se hace la pregunta: ¿Qué clave resultaría si el email estuviera acá?. Si la clave resultante tiene valores válidos, esa posición es candidata. Además, como la clave se repite cada 16 bytes, si una misma posición de la clave aparece varias veces, debería dar siempre el mismo valor. Si da valores contradictorios, esa posición se descarta. Entonces, cuando se encuentra una ubicación coherente, se puede reconstruir la clave completa de 16 bytes.

Sabemos además que los bytes de la clave no pueden tomar cualquier valor. La clave efectiva surge del XOR entre caracteres hexadecimales, y los caracteres hexadecimales posibles son solo:

- 0 1 2 3 4 5 6 7 8 9 a b c d e f

Es por eso que los posibles bytes de clave están limitados, lo cual permite descartar muchas claves falsas. Si al probar una posición aparece un valor de clave imposible, esa posición no sirve.

Finalmente, una vez obtenida la clave de 16 bytes, se repite esa clave a lo largo de todo el texto cifrado. Luego se aplica XOR entre el cifrado y la clave repetida:

- texto cifrado XOR clave reducida = texto claro

Todo esto es posible porque la operación XOR es reversible, entonces eso hace posible recuperar el texto claro original. Y el resultado final debería ser un texto ASCII legible con formato de email.

2.2. Implementación de la solución

El trabajo práctico fue desarrollado en el lenguaje de programación Python, trabajando en un entorno virtual para incluir las librerías y dependencias necesarias para el funcionamiento. EL código del trabajo se encuentra versionado en Git, contenido en un repositorio de Github cuyo enlace de acceso es el siguiente:

- <https://github.com/santiago-vietto/Trabajos-Practicos-Criptografia>

2.2.1. Conexión con el servidor, obtención del desafío y decodificación Base64

Como primer paso de la resolución de la actividad se prepara la información necesaria para comunicarse con el servidor del práctico y solicitar el mensaje cifrado correspondiente al correo registrado.

1) Se define el correo electrónico registrado, en este caso `santiagoviettpo5@gmail.com` el cual pertenece al alumno que lleva a cabo el presente trabajo. Además, se convierte dicho correo a bytes ASCII, porque más adelante se lo usará como texto claro conocido. Esto es importante porque, como vimos antes, la consigna indica que el mensaje descifrado es un correo dirigido a esa dirección, por lo que podemos asumir que el email aparece dentro del mensaje original.

2) Se define la dirección base del servidor y se arma la URL del desafío. Esta URL es la que permite pedir el texto cifrado asociado al email declarado.

3) Se realiza la solicitud al servidor mediante un GET, pasando por parámetro la URL. Luego, la respuesta se guarda como bytes, porque las operaciones que se realizarán posteriormente se hacen sobre secuencias de bytes.

```
8 email = "santiagovietto5@gmail.com"
9 known_plaintext = email.encode("ascii")
10
11 server = "https://cripto.iua.edu.ar"
12 url_challenge = f'{server}/md5crypt/{email}/challenge'
13
14 response = requests.get(url_challenge)
15
16 challenge = response.content
17
18 print("\nRespuesta servidor:")
19 print(challenge.decode('ascii'))
20
21 ciphertext = base64.b64decode(challenge)
```

4) Se imprime el desafío recibido para verificar que el servidor respondió correctamente. Lo que se muestra a continuación no es el mensaje original, sino una cadena codificada en Base64:

```
(venv_TPsCrypto) santiago-vietto@santiago-vietto-pc:~/Documents/Trabajos-Practicos-Criptografia$ python T
P1_Santiago_Vietto.py

Respuesta servidor:
AVICdwNhuJhTYg04BCU0ZVMLUUVUawN7AiABcFluV2VwXFcUXj0COFMnBwQLLVNnUXVSIaFtBXoBJAhpACsEEwFkAnoDYLI/U3cNNwQ0D
nFTZLFsVGkDNwJeAUNZcldvVjtXbV5+AldTdAc0CyxTILE7UnUBIgVqASUITAA8BCsBYAJvA3NSP1NiDXUEmg4wU2hRPVQ0A0oC0wFrWx
RXZVY4VynecwJ2U34HIQs7UzhrJ1J0ATQFdwEjCCMAKQQ/AWACawNtUn5TJw04BDkOPLN3UXBUYQN9AmkBcFL0V2ZWe1dvXLQCVLNoB2s
LflNxlWZSbgELBwYBNghrADYEJQFoAmcDd1ImU2gNbgQRDjhTaFFiVG0DZQJ6AWZZb1dtVLxXE14/AnZTYgdrC35TVLFyUmUBfQUvAwVI
NAB5BBkBYAJsAyNSYFM3DwKEyW5/UzdRMLQ+AzsCYQE/WTFXMVZ2V3xebgIyUzcHYQtUUhRU1JoATQFfQEYCCwAMAQgASECbQNTUj5Tf
g17BD40MVNgUSNUcwNmAiYBYVkgV2ZW0VcLXn4CY1NuBzULfLN2UW9SYQE1BS8BPgh/AHkENAFkAmwDdLI7U2kNPgQ9DiZJVF0VG0DfQ
I8AwPzdVd0VnZXJF4qAnBTbgc/CzLTcVErUgoBMAVhATMILAAAtBDsBYAJ2AyNSJVNoDSkENQ5/U2xRcFQkA2sCOAFkWWNXa1Y7VzZeNwJ
uUykHwrtXUwtRKLIItAXEFTAE4CGAANARzAUMCcAnsUjVTZg01
```

5) Finalmente, se decodifica el Base64 para obtener los bytes cifrados reales. Como vimos antes, Base64 no es un cifrado, sino una forma de representar bytes como texto, es por eso que aca solo se transforma la respuesta del servidor en el verdadero contenido cifrado sobre el cual se va a trabajar.

2.2.2. Eliminación de clave auxiliar interna y reducción del texto cifrado

En este paso se toma el texto cifrado real, obtenido luego de decodificar Base64, y se lo transforma en un nuevo cifrado más simple.

1) Se crea una variable vacía de tipo bytes en donde se va a guardar el nuevo texto cifrado reducido.

2) El ciclo for recorre el texto cifrado de a dos bytes. En cada vuelta toma un byte de posición par y el byte siguiente de posición impar. Luego aplica XOR entre ambos:

- byte_par XOR byte_impar

El resultado se agrega al nuevo texto cifrado reducido. El texto todavía queda afectado por la clave original, pero la clave auxiliar interna ya no participa.

```
27 reduced_ciphertext = b""
28
29 for i in range(0, len(ciphertext), 2):
30     byte_par = ciphertext[i]
31     byte_impar = ciphertext[i + 1]
32
33     resultado = byte_par ^ byte_impar
34
35     reduced_ciphertext += bytes([resultado])
36
37 print("\nCantidad de bytes del texto cifrado original:", len(ciphertext))
38 print("Cantidad de bytes del texto cifrado reducido:", len(reduced_ciphertext))
39
```

3) Finalmente, se imprime la longitud del texto cifrado original y la longitud del texto reducido. El reducido debería tener aproximadamente la mitad de bytes, porque de cada par de bytes originales se obtiene un solo byte nuevo.

```
(venv_TPsCrypto) santiago-vietto@santiago-vietto-pc:~/Documents/Trabajos-Practicos-Criptografia$ python T
P1_Santiago_Vietto.py
Cantidad de bytes del texto cifrado original: 588
Cantidad de bytes del texto cifrado reducido: 294
```

2.2.3. Obtención de los posibles bytes válidos de la clave reducida.

En este paso se define qué valores puede tener la clave que quedó después de reducir el cifrado. La idea es reducir las posibilidades de búsqueda. Si no supiéramos nada de la clave, cada byte podría tener hasta 256 valores posibles. Pero por la forma en que está construido el algoritmo, sabemos que los bytes válidos son muchos menos. Entonces, cuando más adelante se intente recuperar la clave usando el texto conocido, podremos descartar rápidamente cualquier valor que no pertenezca a este conjunto de posibles valores.

1) Se define el conjunto de caracteres hexadecimales posibles. Esto se hace así ya que, como las claves internas derivan de MD5, y el resultado de este algoritmo está representado como texto hexadecimal, por eso sus caracteres posibles son solamente: 0 1 2 3 4 5 6 7 8 9 a b c d e f.

2) Se crea un conjunto vacío en donde se van a guardar todos los valores posibles que puede tomar un byte de la clave reducida. Se usa set() porque no interesa repetir valores, ya que solo queremos saber cuáles son válidos.

3) En el ciclo for se prueban todas las combinaciones posibles entre dos caracteres hexadecimales. Y cada resultado se almacena en el conjunto vacío.

La razón es que, después de eliminar la clave interna, los bytes de la nueva clave resultan del XOR entre caracteres hexadecimales. Por eso no cualquier byte puede formar parte de la clave, sino solamente aquellos que puedan obtenerse de:

- carácter hexadecimal XOR carácter hexadecimal

Por ejemplo:

0 XOR 0

0 XOR 1

0 XOR 2

...

```
41
42 # Paso 3: Obtencion de los posibles bytes validos de la clave efectiva.
43
44 hex_digits = b"0123456789abcdef"
45 valid_key_bytes = set()
46
47 for a in hex_digits:
48     for b in hex_digits:
49         valid_key_bytes.add(a ^ b)
50
51 print("\nValores de bytes validos: ", valid_key_bytes)
52
```

4) Finalmente, se imprimen los valores válidos encontrados. Es decir, la clave no puede tener cualquier byte posible, sino un conjunto limitado de valores.

```
(venv_TPsCrypto) santiago-vietto@santiago-vietto-pc:~/Documents/Trabajos-Practicos-Criptografia$ python T
P1_Santiago_Vietto.py

Valores de bytes validos: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 80, 81, 82, 83, 84, 85,
86, 87, 88, 89, 90, 91, 92, 93, 94, 95}
```

2.2.4. Obtención de la clave reducida y descifrado del texto claro

En este paso se utiliza el texto conocido, es decir el email registrado, para recuperar la clave reducida de 16 bytes y luego descifrar el texto claro.

1) Se define la longitud de la clave reducida en 16. Esto viene del análisis anterior, en donde después de reducir el cifrado, el problema queda como un cifrado tipo Vigenère con XOR, donde la clave se repite cada 16 bytes. Y se define una variable mensaje para saber si finalmente se pudo descifrar algo, por lo que si al terminar sigue siendo None, significa que no se encontró una clave reducida válida.

2) Probamos todas las posiciones posibles del email. Nosotros no sabemos en qué parte del mensaje aparece el email registrado, puede aparecer al principio, al medio, al final, etc. Entonces el código prueba todas las posiciones posibles dentro del texto cifrado reducido. La variable offset representa “supongamos que el email empieza en esta posición”, y por cada offset se crea un diccionario en donde se va a guardar los bytes de clave que se van descubriendo.

3) En el ciclo for interno se calculan los bytes de la clave usando el texto conocido (email registrado), en donde se recorre cada byte de este último. Sabiendo que:

- texto claro XOR clave = texto cifrado

Entonces podemos despejar la clave:

- $\text{texto cifrado XOR texto claro} = \text{clave}$

Es por eso que se hace el siguiente cálculo en el que se obtiene un posible byte de la clave:

- $\text{valor_clave} = \text{reduced_ciphertext}[\text{offset} + i] \wedge \text{known_plaintext}[i]$

Y la operación declarada en `posicion_clave` sirve para ubicar ese byte dentro de la clave de 16 posiciones. Como la clave se repite (0, 1, 2 ..., 15, 0, 1,...) se usa módulo 16.

4) Se descartan los valores de clave imposibles. Se revisa si el byte de clave calculado pertenece al conjunto de valores válidos que habíamos generado antes. Si no pertenece, ese offset no sirve. Es decir, si al suponer que el email empieza en una posición, aparece un byte de clave imposible, entonces el email no empieza en dicha posición. Por eso se corta con `break` y se prueba otro offset.

5) Se descartan contradicciones. Como la clave tiene solo 16 bytes y se repite, una misma posición de clave puede aparecer más de una vez al analizar el email. Por ejemplo, la posición 3 de la clave debería tener siempre el mismo valor. Si en una parte del email se calcula `clave[3] = 80` pero más adelante da `clave[3] = 92` entonces hay una contradicción. Eso significa que el offset probado no puede ser correcto.

6) Se guarda el byte de la clave encontrado. Si el valor es válido y no contradice nada, se guarda en el diccionario. El diccionario quedaría conceptualmente como:

- Posición 0 de la clave -> valor encontrado
- Posición 1 de la clave -> valor encontrado
- Posición 2 de la clave -> valor encontrado
- ...

7) En el condicional se verifica si se encontró la clave completa. Se comprueba si ya se encontraron las 16 posiciones de la clave. Entonces, si `posible_clave` tiene 16 elementos, significa que se logró reconstruir la clave completa. Esto es posible porque el email registrado tiene más de 16 caracteres, entonces alcanza para cubrir todas las posiciones de la clave repetida.

8) Se construye la clave efectiva. Se crea una clave vacía de 16 bytes. Luego se rellenan sus posiciones con los valores encontrados. Y finalmente se convierte a bytes. Conceptualmente sería:

- `clave[0] = valor encontrado`
- `clave[1] = valor encontrado`
- ...
- `clave[15] = valor encontrado`

Después de esto, ya tenemos la clave efectiva de 16 bytes. Recordamos que esta no es la contraseña original del usuario. Es la clave efectiva que sirve para descifrar este mensaje reducido.

9) Ahora se usa la clave encontrada para descifrar todo el texto reducido. Como el cifrado reducido era:

- `mensaje_original XOR clave_repetida = reduced_ciphertext`

Para recuperar el mensaje original hacemos:

- `reduced_ciphertext XOR clave_repetida = mensaje_original`

Y se hace que la clave se repita cada 16 bytes.

Además, el resultado descifrado está en bytes. Pero como la actividad dice que el texto claro está codificado en ASCII, se convierte a texto claro descifrado usando ASCII.

```
57 key_len = 16
58 mensaje = None
59
60 for offset in range(0, len(reduced_ciphertext) - len(known_plaintext)):
61 |
62 |     posible_clave = {}
63 |
64 |     for i in range(len(known_plaintext)):
65 | |
66 | |         posicion_clave = (offset + i) % key_len
67 | |         valor_clave = reduced_ciphertext[offset + i] ^ known_plaintext[i]
68 | |
69 | |         if valor_clave not in valid_key_bytes:
70 | | |             break
71 | |
72 | |         if posicion_clave in posible_clave and posible_clave[posicion_clave] != valor_clave:
73 | | |             break
74 | |
75 | |         posible_clave[posicion_clave] = valor_clave
76 |
77 | if len(posible_clave) == key_len:
78 | |
79 | |     key = bytearray(b"\x00" * key_len)
80 | |
81 | |     for posicion, valor in posible_clave.items():
82 | | |         key[posicion] = valor
83 | |
84 | |     key = bytes(key)
85 | |
86 | |     plaintext = b""
87 | |
88 | |     for j in range(len(reduced_ciphertext)):
89 | | |         plaintext += bytes([reduced_ciphertext[j] ^ key[j % key_len]])
90 | |
91 | |     mensaje = plaintext.decode("ascii", errors="replace")
92 | |
93 | |     print("\nOffset encontrado:", offset)
94 | |     print("Clave encontrada:", key)
95 | |
96 | |     print("\nMensaje descifrado:")
97 | |     print(mensaje)
98 | |
99 | |     break
```

```

100
101
102 if mensaje is None:
103     print("\nNo se pudo descifrar el mensaje.")
104     exit()
105
106

```

11) Los resultados que se muestran son:

- La posición donde se encontró el email registrado.
- La clave efectiva encontrada.
- EL texto claro descifrado.

```

Offset encontrado: 119
Clave encontrada: b'\x00\x00\x00\x00TVUQVRP\nV\x04YW'

Mensaje descifrado:
Subject: Fortune
Cc: User <user@example.com>
From: User <user@example.com>
Content-type: text/plain, charset=utf-8
To: santiagovietto5@gmail.com
Date: Tue, 18 Jan 2022 21:25:11 +0000

There is only one word for aid that is genuinely without strings,
and that word is blackmail.
-- Colm Brogan

```

2.2.5. Envío de la respuesta al servidor

Una vez obtenido el texto claro descifrado, el objetivo ahora es enviarlo al servidor para que valide si el desafío fue resuelto correctamente.

- 1) Se crea la URL que apunta al endpoint donde el servidor espera recibir el mensaje descifrado.
- 2) Luego, se realiza una petición POST en la que se envía el mensaje descifrado usando el campo de formulario llamado message. Esto es importante porque la consigna indica que la respuesta debe mandarse como contenido de tipo formulario, con un campo message. Además, se simula el envío de un archivo de texto llamado message.txt, cuyo contenido es el mensaje descifrado, que equivale conceptualmente al ejemplo con curl.

```

111 answer_url = f"{server}/md5crypt/{email}/answer"
112
113 response_answer = requests.post(
114     answer_url,
115     files={
116         "message": ("message.txt", mensaje, "text/plain")
117     }
118 )
119
120 print("\nRespuesta del servidor:")
121 print("Código:", response_answer.status_code)
122 print(response_answer.text)
123
124

```

3) Finalmente, se imprime la respuesta del servidor, lo que permite ver si el envío fue correcto, mostrando el código de estado de la petición http al servidor, por ejemplo, código 200 como en este caso que salió bien, y el contenido de response_answer.text que muestra el mensaje que devuelve el servidor, en este caso ¡Ganaste!, indicando que la actividad fue resuelta correctamente.

```
(venv_TPsCrypto) santiago-vietto@santiago-vietto-pc:~/Documents/Trabajos-Practicos-Criptografia$ python T
P1_Santiago_Vietto.py
-
Respuesta del servidor:
Código: 200
¡Ganaste!
```